BERT Note 1st

1. Transformer
2. BERT
3. Project: Paper, Code

Part 1 Transformer

在"Atten is all you need"中，首次提出 Transformer 架构，其只通过注意力机制来计算模型输入与输出的隐含表示，即自注意力机制（**self-attention**），如图的 Multi-Head Attention。

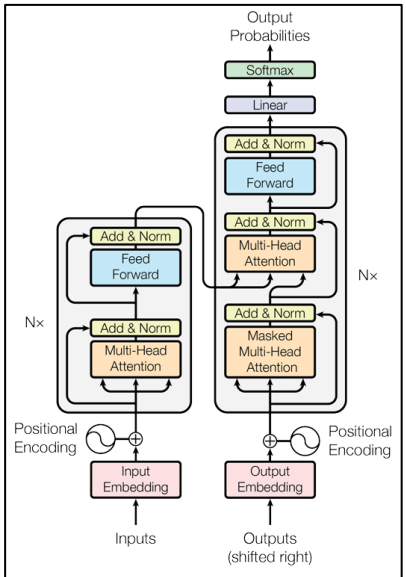


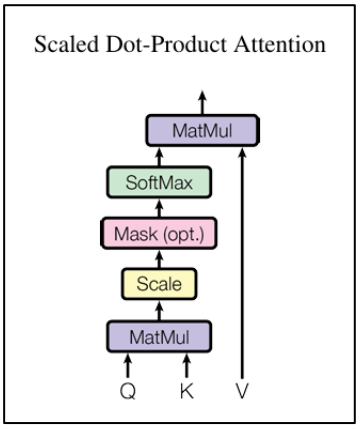
Figure 1: The Transformer - model architecture.

> "To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence- aligned RNNs or convolution."

总体来说，**所谓自注意力机制就是通过某种运算来直接计算得到句子在编码过程中每个位置上的注意力权重；然后再以权重和的形式来计算得到整个句子的隐含向量表示。**最终，Transformer 架构就是基于这种的自注意力机制而构建的 Encoder-Decoder 模型。

Self-attention

Attention 函数是从一个序列(Query)和一个键值对集合(Key-Value Pairs Set)到一个输出的映射，通过 queries 和所有 keys 的点积，除以$\sqrt{\boldsymbol{D_k}}$，再应用一个 Softmax 函数获取 values 的权重。



> "An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors.
>
> The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key."

文中提出的 Scaled Dot-Product Attention, 输入包括 Queries, 维度为d_k的 Keys 和维度为d_v的 Values。 将 Queries, Keys & Values 集合成矩阵，用来计算 Attention, 公式为:

$$\boldsymbol{Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{D_k}}\right)V} \tag{1}$$

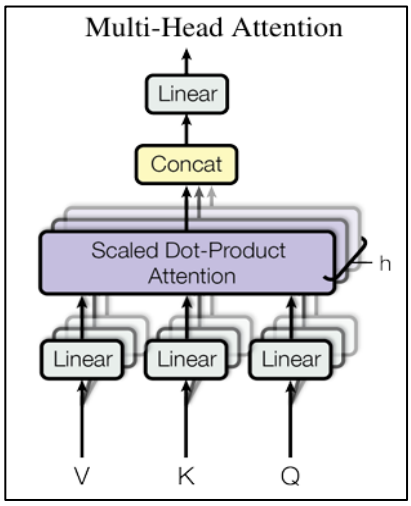
其中$Q \in \mathbb{R}^{n\times d_k}$，$Q$的每一行为一个 Query, $K \in \mathbb{R}^{n\times d_k}, V \in \mathbb{R}^{n\times d_v}$。除以$\sqrt{d_k}$为一个归一化因子，避免点积的值过大导致 Softmax 之后的梯度过小。

值得注意，模型对当前位置的信息编码时，会过度集中注意力于自身位置（虽符合常识）而可能忽略了其它位置。因此，解决方案就是多头注意力机制（Multi-Head Attention）。

> "It expands the model's ability to focus on different positions. Yes, in the example above, contains a little bit of every other encoding, but it could be dominated by the actual word itself.
>
> Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions."

Multi-Head Attention

作者提出了通过多头注意力机制来解决这一问题。**同时，使用多头注意力机制还能够给予注意力层的输出包含有不同子空间中的编码表示信息，从而增强模型的表达能力。**



> 学会接受层的概念，每个层里面封装了一段数据处理的逻辑。将多层叠加和组合，奇妙的化学反应便由此而生。

Multi-Head Attention 并不直接对原始的 Keys, Values 和 Queries 应用注意力函数，而是学习一个三者各自的映射再应用 Attention，同时将这个过程重复h次。具体的计算公式为:

$$MultiHead(Q,K,V) = Concat(head_1, \dots, head_h)W^O$$

$$where\ head_i = Attention\left(QW_i^Q, KW_i^K, VW_i^V\right) \tag{2}$$

其中$W_i^Q \in \mathbb{R}^{d_{model}\times d_q}, W_i^K \in \mathbb{R}^{d_{model}\times d_k}, W_i^V \in \mathbb{R}^{d_{model}\times d_v}, W_i^O \in \mathbb{R}^{hd_v\times d_{model}}$,

从这里可发现，论文中多头注意力机制其实就是将一个大的高维单头拆分成h个多头。

$h = 8$为重复次数，则有$d_k = d_v = \frac{d_{model}}{h} = 64$。

Embedding: Positional Embedding

DL 将各个词或字通过 Embedding 层映射到低维稠密的向量空间。因此，Transformer 将文本的向量化表示称为 **Token Embedding**, 即 **Word Embedding(词嵌入)**。

根据 self-attention 原理的介绍，自注意力机制在实际运算过程中不过就是几个矩阵来回相乘进行线性变换而已。因此，这就导致即使是打乱各个词的顺序，最终计算得到的结果本质上却没有任何变换，即仅仅**只使用自注意力机制会丢失文本原有的序列信息。**

基于上述原因，Transformer 在原始输入文本进行 Token Embedding 后，又额外的加入了一个 Positional Embedding 来刻画数据在时序上的特征。

> "Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence."

在 Transformer 中，作者采用了如公式所示的规则来生成各个维度的位置信息，

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \tag{3}$$

Positional Embedding 矩阵中，$pos \in [0, \max len]$表示某一位置, $i \in \left[0, \frac{d_{model}}{2}\right]$表示某一维度。选用这种表示形式的原因是对于固定的某一偏移k, PE_{pos+k}都可用PE_{pos}线性表示。

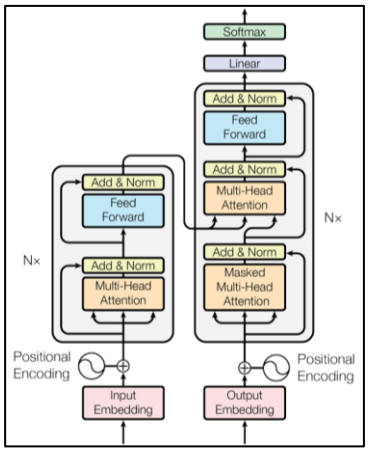
$$\sin(\alpha+\beta) = sin\alpha\ cos\beta + cos\alpha\ sin\beta$$
$$\cos(\alpha+\beta) = cos\alpha\ sin\beta - sin\alpha\ cos\beta$$

在交换位置前与后，与同一个权重矩阵进行线性变换的结果截然不同。因此，证明了通过 Positional Embedding 可以弥补自注意力机制不能捕捉序列时序信息的缺陷。

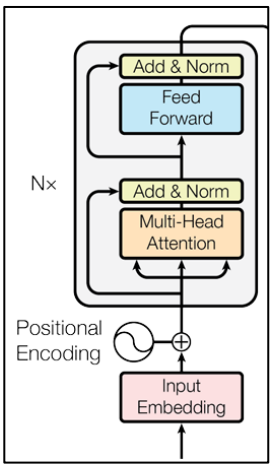
Transformer Layer

如右图便是一个单层 Transformer 网络结构图。

整个 Transformer 网络包含左右两个部分，即 Encoder 和 Decoder。



Encoder



首先，对于 Encoder，其网络结构如上图左侧所示（尽管论文中是以 6 个这样相同的网络层构成，每层中两个子层，但这里先介绍一层的堆叠，多层结构稍后）。

如图，Encoder 内部由两部分网络所构成：**Multi-Head Attention 层和 Position-Wise 全链接的前馈神经网络。**

> "The encoder is composed of a stack of N = 6 identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network."

同时，每一层应用残差连接(Residual Connection)和层归一化(Layer Normalization)。每部分输出均为$\mathrm{LayerNorm}(\mathrm{x}+\mathrm{Sublayer}(\mathrm{x}))$，其中$\mathrm{Sublayer}(\mathrm{x})$为子层本身的函数实现。

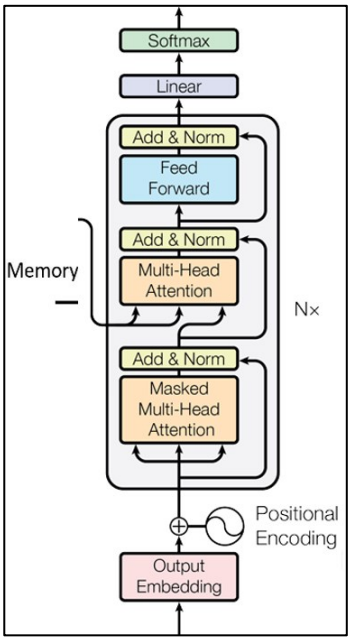
进一步，为了便于在这些地方使用残差连接，这两部分网络输出向量的维度均为$d_{model}=512$。对于第 2 部分的两层全连接网络来说，其具体计算过程为

$$FFN(x)=\max(0,xW_1+b_1)\,W_2+b_2 \tag{4}$$

其中输入x的维度为$d_{model}=512$，第 1 层全连接层的输出维度为$d_{ff}=2048$，第 2 层全连接层的输出为$d_{model}=512$，**且对第 1 层网络的输出还运用了 Relu 激活函数。**

Decoder

同于 Encoder，论文也采用 6 个完全相同的网络层堆叠而成，这里依旧只先看 1 层时。**对于 Decoder，其整体类似 Encoder，只是多一个与 Encoder 输出交互的多头注意力机制**，如图。



不同 Encoder，Decoder 共包含 3 个网络结构的部分。**最上面和最下面的部分（暂时忽略 Mask）与 Encoder 相同，只是多了中间这个与 Encoder 输出（Memory）进行交互的部分**，作者称之为"Encoder-Decoder attention"。

对于这部分的输入，**Q 来自于下面多头注意力机制的输出，K 和 V 均是 Encoder 部分的 output（Memory）经过线性变换后得到**。该设计模仿传统 Encoder-Decoder 网络模型 seq to seq 的解码过程。

Part 2: BERT

Language model pre-training 对于下游很多自然语言处理任务都有着显著的改善。但现有预训练模型的网络结构限制了模型本身的表达能力，尤其对于 fine-tuning approaches. **其中最主要的限制是采用 unidirectional 的方法来对输入进行编码。**

例如 OpenAI GPT 使用了 left-to-right 的网络结构，就使得模型在编码过程中只能看到当前时刻之前的信息，而不能同时捕捉到当前时刻之后的信息。

论文用 BERT (Bidirectional Encoder Representations from Transformers) 来实现模型的双向编码学习能力。**同时，模型为了有效学习，BERT 在训练中使用 Masked Language Model (MLM)，即随机遮蔽输入序列中的某些位置，然后通过模型来对其预测。**

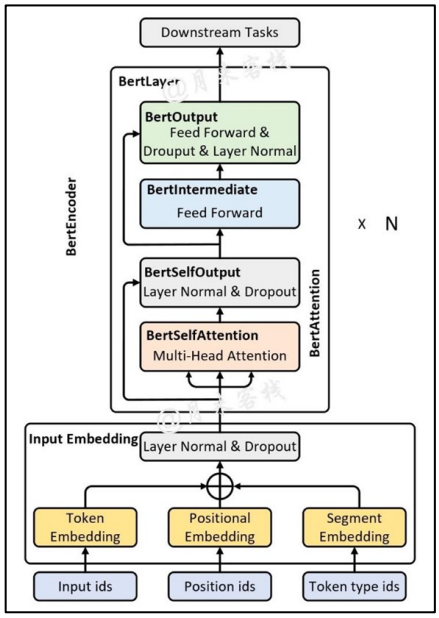
由于 MLM 预测任务能使模型编码的结果同时包含上下文的语境信息，因此利于训练得到更深的 BERT 网络模型。**此外，训练 BERT 的过程中作者还加入了 Next Sentence Prediction (NSP)，即同时输入两句话到模型以预测第 2 句话是否为第 1 句的下一句话。**

BERT Network Structure

BERT 网络结构整体上就是由多层的 Transformer Encoder 堆叠形成。

如图可发现其上半部分结构与 Transformer Encoder 类似，**只不过在 Input 多了 Segment Embedding。**

接下来分别对各个部分介绍。



Input Embedding

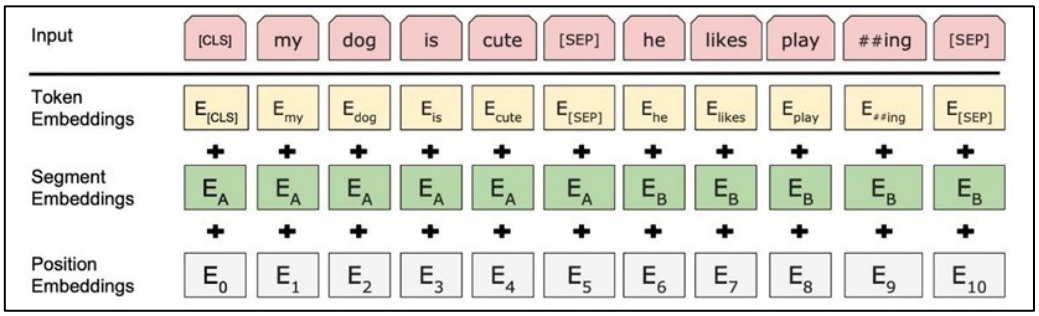
BERT 该模块包含三个部分: **Token Embedding, Positional Embedding 和 Segment Embedding**。

注意，BERT 中 PE 对于每个位置的编码并不是公式计算得到，而是类似普通的 Word Embedding 一样为每个位置初始化了一个向量，然后随着网络一起训练得到。

值得一提的是，**BERT 开源的预训练模型最大只支持 512 个字符的长度，这是因为其在训练过程中（位置）词表的最大长度只有 512。**

除此之外，第三部分则是 Segment Embedding。由于需要构建一个通用的预训练模型，因此要兼顾到各种 NLP 任务场景下的输入。**Segment Embedding 的作用便是区分输入序列中的不同部分，其本质就是通过一个普通的词嵌入来区分每一个序列所处的位置。**

最后，再将这三部分 Embedding 后的结果相加（并进行标准化）便得到了最终的 Input Embedding 部分的输出，如图。



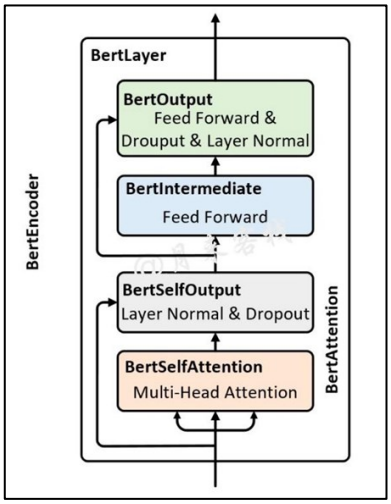
Input 表示输入序列，其中字符“[CLS]”是指特殊的分类标志，如果下游任务是做文本分类，那么 BERT 的输出结果只取“[CLS]”对应的向量进行分类即可（不过实验表明，取所有位置向量的均值往往效果更好）；**而“[SEP]”字符则用来作为将两句话分开的标志。**

Segment Embedding 层则同样是用来区分两句话所在的不同位置，对于每句话来说其内部各自的位置向量都是一样的。如果原始输入只有一句话，那么 Segment Embedding 层中对应的每个 Token 的位置向量都相同。

最后，Positional Embedding 则是用来标识句子中每个 Token 各自所在的位置，使得模型能够捕捉到文本“有序”这一特性。

Bert Encoder

如图为 Bert Encoder 结构示意图，整体由多个 Bert Layer（论文中指代 Transformer blocks）构成。



具体的，作者分别用 **L 来表示 Bert Layer 的层数**，即 Bert Encoder 是由 L 个 Bert Layer 所构成；**用 H 表示模型的维度；用 A 表示多头注意力中多头的个数**。

同时，论文分别就 $BERT_{BASE}(L=12, H=768, A=12)$ & $BERT_{LARGE}(L=24, H=1024, A=16)$ 两种尺寸的 BERT 模型进行了实验对比。

Masked Language Model & Next Sentence Prediction

为更好地训练 BERT 网络，作者在 BERT 的训练过程中引入 MLM 和 NSP。对于 MLM 任务来说，其做法是随机掩盖掉输入序列中 15%的 Token（即用“[MASK]”替换原有的 Token），然后在 BERT 的输出结果中取对应掩盖位置上的向量进行真实值预测。

接着，作者提到虽然 MLM 的这种做法能够得到一个很好的 pre-training 模型，但仍存在不足。**在 fine-tuning 时，由于输入序列中并不存在“[MASK]”这样的 Token，将导致 pre-training 和 fine-tuning 之间存在不匹配不一致的问题（GAP）。**

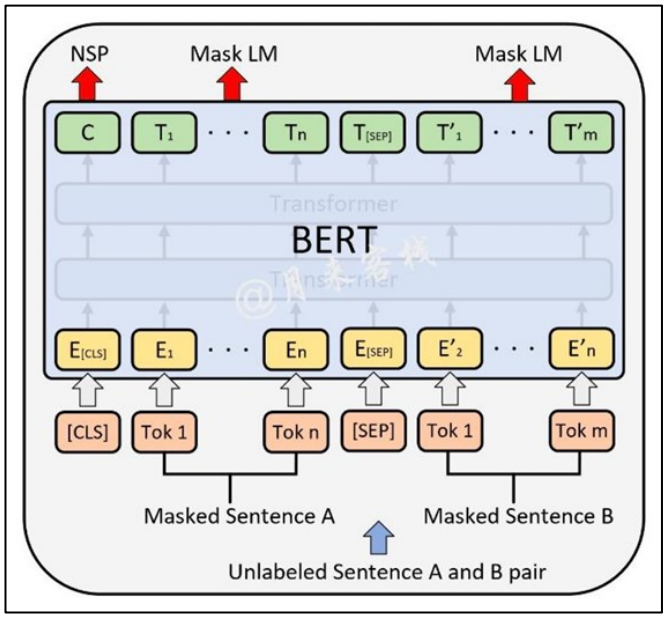
为解决此问题，作者在原始 MLM 的基础上做了部分改动，即先选定 15%的 Token，然后将其中的 80%替换为“[MASK]”、10%随机替换为其它 Token、剩下的 10%不变。最后取这 15%的 Token 对应的输出做分类来预测其真实值。

> “The training data generator chooses 15% of the token positions at random for prediction. If the i-th token is chosen, we replace the i-th token with (1) the [MASK] token 80% of the time (2) a random token 10% of the time (3) the unchanged i-th token 10% of the time.”

又由于，很多下游任务需要依赖于分析两句话之间的关系来进行建模，例如问题回答等。**为使模型能具备该能力，作者在论文中又提出了二分类的下句预测任务 NSP。**

具体地，每个样本由 A 和 B 两句话构成，其中 50%情况 B 为 A 的下一句话（标签为 IsNext），另外 50%情况 B 是语料其它的随机句子（NotNext），然后模型预测 B 是否为 A 的下一句话。

如图所示便是 MLM 和 NSP 这两个任务在 BERT 预训练时的输入输出示意图，**其中最上层输出的 C 在预训练时用于 NSP 中的分类任务；其它位置的T_i, T'_j则用于预测被掩盖的 Token.**



Project: Faithful BERT

Ma: 核心关注 text 方面的 faithfulness，实现在 synonym substitution 和(potentially, yet to be started) word reordering 情况下的 robustness。目前在做代码 denoising 方面的尝试.

Paper 1 SEAT: Stable and Explainable Attention

最近的研究表明 **attention 因训练或测试时的 randomness 和 perturbation 而不稳定** 比如随机种子和 embedding 的轻微 perturbation 妨碍其成为 faithful 解释工具。论文中提出一种严格定义的 attention method 为 SEAT 并且具有三种属性:

1. **其预测分布接近 vanilla attention[1]的预测**
2. **前 k 项 indices 很大程度上与 vanilla attention 重叠**
3. **关于 perturbations 很 robust**

此外 我们为获得 SEAT 提出一种优化方法 被认为改进了 vanilla attention。最终 我们使用不同的评估矩阵比较了 RNN BiLSTM and BERT 架构 在解释, 稳定和精确性方面。

结果表明 除了保留了原有的解释性和模型性能 SEAT 针对 randomness 和 perturbations 更稳定 意味着它是更 faithful 的解释模型

Introduction

注意力机制作为一种 faithful interpretation 被怀疑是不稳定的 此外 在推断时对于输入的 perturbations 同样脆弱 并且扰乱 word embeddings 同样影响预测分布和可解释性

实际上 **不稳定性被普遍认为是 deep models 的 interpretation methods 中的问题** 为了解决上述难题 我们先给出严格定义 直觉上 稳定的 attention 有如下性质:

1. **产生相似预测来确保模型的效用**
2. **Top-k indices 中 stable 注意力要与 vanilla 注意力大量重叠 使前者继承可解释性**
3. **对于 randomness in training 或 input perturbations in testing 足够 robust**

并且我们提供如下 SEAT 的数学定义来保持正确性:

1. **SEAT 确保 loss 在预测分布之间充足地小**
2. **确保 SEAT 与 vanilla attention 在 top-k indices 的重叠足够大**
3. **SEAT 保证 perturbations 对其的预测分布的改变够小**

---

[1] query 来自于解码层，key 和 value 来自于编码层时叫 vanilla attention，即最基本的 attention。而 query, key 和 value 都来自编码层的叫 self-attention。

论文第二部分则**提出一种找寻 SEAT 的方法 具体地 将呈现一种 min-max stochastic optimization 问题 其目标函数涉及三个参数** 然而 难点是性质 2 中的参数 non-differentable 所以我们**又提出了替代 top-k overlap function 的损失函数**

最终 我们**大规模实验 RNN, BiLSTM 和 BERT 来验证 SEAT 的三种性质** 尤其通过三种 perturbations or randomness: random seeds, embedding vector perturbation, and input token perturbation. **三种评估矩阵也被使用于模型可解释性**

结果表现, SEAT 是一种 more faithful interpretation. 此外 通过比较 F1 score 发现 SEAT 相比于 vanilla attention 几乎没有 accuracy degradation.

Stable and Explainable Attention

Vanilla Attention. 我们先简单介绍注意力机制 记$x \in \mathbb{R}^{s\times|V|}$为模型在每个位置由 one-hot 编码词组成的输入 存在维度为d的 embedding 矩阵E 在输入矩阵后 单词具有更密集的 token representations $x_e \in \mathbb{R}^{s\times d}$.

对于 **Enc**(encoder), 它有序读入 embedded tokens 并产生 s number of m-dimensional hidden states, i.e., $\boldsymbol{h}(x) = \boldsymbol{Enc}(x_e) \in \mathbb{R}^{s\times m}$, $h_t(x)$作为表示在 x 中位于位置 t 的词

A similarity function ϕ映射$h(x)\ and\ query\ \boldsymbol{Q} \in \mathbb{R}^m$ to scalar scores, 并且注意力权重为 $\boldsymbol{w}(x) = softmax\big(\phi(\boldsymbol{h}(x), \boldsymbol{Q})\big) \in \mathbb{R}^s$. 我们考虑两种常见的 similarity functions:

1. $Additive\ \phi(\boldsymbol{h}(x), \boldsymbol{Q}) = \boldsymbol{v}^T \tanh(\boldsymbol{W_1}\boldsymbol{h}(x) + \boldsymbol{W_2}\boldsymbol{Q})$
2. $Scaled\ Dot-Product\ \phi(\boldsymbol{h}(x), \boldsymbol{Q}) = \frac{\boldsymbol{h}(x)\boldsymbol{Q}}{\sqrt{m}}$

Where $\boldsymbol{v}, \boldsymbol{W_1}, and\ \boldsymbol{W_2}$ 是模型参数

基于$\boldsymbol{w}$, 模型在 **Dec**(decoder)过程后的预测, $i.e., y(x, \boldsymbol{w}) = \sigma(\theta \cdot h_w) \in \mathbb{R}^{|Y|}$, where $h_w = \sum_{t=1}^{s} \boldsymbol{w}_t(x) \cdot h_t(x)$, σ 是输出激活函数, θ是参数并且$|Y|$是 number of classes.

Definition 1: Top-k overlaps

For a vector we define the set of top-k component$x \in \mathbb{R}^d, T_k(\cdot)$ as follow,

$$T_k(x) = \{i: i \in [d] and \{|x_j \geq x_i: j \in [d]\}| \leq k\}\}.$$

And for two vectors x, x', the top-k overlap function $V_k(x, x')$ is defined by the overlap ratio between the top-k components of two vectors, $i.e., V_k(x, x') = \frac{1}{k}|T_k(x) \cap T_k(x')|$.

Definition 2: Stable and Explainable Attention

We call a vector $\widetilde{\boldsymbol{w}}$ is $(D_1, D_2, R, \alpha, \beta, \gamma, V_k)$ -Stable and Explainable Attention $w.r.t.$ the vanilla attention $\boldsymbol{w}$ if it is satisfies the following properties for any x:

1. Closeness of Prediction

$$D_1\big(y(x, \widetilde{\boldsymbol{w}}), y(x, \boldsymbol{w})\big) \leq \gamma \; for \; \gamma \geq 0,$$

where D_1 is some divergence function, $y(x, \widetilde{\boldsymbol{w}}) = \sigma(\theta \cdot h_{\widetilde{w}}) \in \mathbb{R}^{|Y|}$ and $y(x, \boldsymbol{w}) = \sigma(\theta \cdot h_w) \in \mathbb{R}^{|Y|}$

2. Similarity of Explainability

$$V_k\big(\widetilde{\boldsymbol{w}}(x), \boldsymbol{w}(x)\big) \geq \beta \; for \; some \; 1 \geq \beta \geq 0;$$

3. Stability

$$D_2\big(y(x, \widetilde{\boldsymbol{w}}), y(x, \widetilde{\boldsymbol{w}} + \delta)\big) \leq \alpha \; for \; all \; \big|\big|\delta\big|\big| \leq R,$$

Where D_2 is some divergence function, $||\cdot||$ is a norm and $R \geq 0$.

Finding a SEAT

为了找寻 SEAT 我们公式化一个 min-max optimization problem, 以第一个条件作为目标且剩下两个作为约束 具体地 首先我们有

$$\min_{\widetilde{\boldsymbol{w}}} \mathbb{E}_x D_1\big(y(x, \widetilde{\boldsymbol{w}}), y(x, \boldsymbol{w})\big) \tag{1}$$

对于任意输入x, 如果没有更多约束 那么公式 1 的最小化就是 vanilla attention w.

接着 我们将约束加入目标函数:

$$\forall x \; s.t. \max_{||\boldsymbol{\delta}|| \leq R} D_2\big(y(x, \widetilde{\boldsymbol{w}}), y(x, \widetilde{\boldsymbol{w}} + \delta)\big) \leq \alpha, \tag{2}$$

$$V_k\big(\widetilde{\boldsymbol{w}}(x), \boldsymbol{w}(x)\big) \geq \beta. \tag{3}$$

结合公式 1-3 并用 regularization 处理约束, 得到以下目标函数:

$$\min_{\widetilde{\boldsymbol{w}}} \mathbb{E}_x[D_1\big(y(x, \widetilde{\boldsymbol{w}}), y(x, \boldsymbol{w})\big) + \lambda_2(\beta - V_k\big(\widetilde{\boldsymbol{w}}(x), \boldsymbol{w}(x)\big)) \\ + \lambda_1(\max_{||\boldsymbol{\delta}|| \leq R} D_2\big(y(x, \widetilde{\boldsymbol{w}}), y(x, \widetilde{\boldsymbol{w}} + \delta)\big) - \alpha)] \tag{4}$$

Where $\lambda_1 > 0 \; and \; \lambda_2 > 0$ 是超参数

由于$V_k\big(\widetilde{\boldsymbol{w}}(x), \boldsymbol{w}(x)\big)$ non-differentiable 阻碍梯度下降 考虑替代的 loss, 见后述

Projected gradient descent to solve $\boldsymbol{\delta}$

We can interpret perturbation as the attack to $\widetilde{\boldsymbol{w}}$ via maximizing δ. Then, δ can be updated by the following procedure in the p-th iteration.

$$\boldsymbol{\delta_p} = \boldsymbol{\delta^*_{p-1}} + \alpha_p \frac{1}{B_p} \sum_{x \in B_p} \nabla D_2(y(x, \widetilde{\boldsymbol{w}}), y(x, \widetilde{\boldsymbol{w}} + \delta^*_{p-1})); \ \boldsymbol{\delta^*_p} = \underset{\|\boldsymbol{\delta}\| \le R}{argmin} \|\boldsymbol{\delta} - \boldsymbol{\delta_p}\|$$

Where α_p is the step size for PGD, B_p is a batch and $|B_p|$ is a batch size.

We **derive the optimal $\boldsymbol{\delta}^*$ in the t-th iteration** for the inner optimization. Specifically, we **find a $\boldsymbol{\delta}$ as the maximum tolerant of perturbation w.r.t $\widetilde{\boldsymbol{w}}$ in the t-th iteration.**

Top-k overlap surrogate loss

Now we seek to a surrogate loss $\mathcal{L}_{Topk}(\widetilde{\boldsymbol{w}}, \boldsymbol{w})\ for - V_k(\widetilde{\boldsymbol{w}}, \boldsymbol{w})$ which can be used in training. We only need ensure high top-k indices overlaps between $\widetilde{\boldsymbol{w}}$ and $\boldsymbol{w}$ constrained on the top-k entries only instead of the whole vectors, $i.e.$,

$$\|\boldsymbol{w}_{S_w^k} - \widetilde{\boldsymbol{w}}_{S_w^k}\|_1$$

where $\boldsymbol{w}_{S_w^k}, \widetilde{\boldsymbol{w}}_{S_w^k} \in \mathbb{R}^k$ is the vector $\boldsymbol{w}$ and $\widetilde{\boldsymbol{w}}$ constrained on the indices set S_w^k respectively and S_w^k is the top-k indices set of $\boldsymbol{w}$. 2 top-k indices sets lead surrogate loss as follows:

$$\mathcal{L}_{Topk}(\widetilde{\boldsymbol{w}}, \boldsymbol{w}) = \frac{1}{2k}(\|\boldsymbol{w}_{S_w^k} - \widetilde{\boldsymbol{w}}_{S_w^k}\|_1 + \|\widetilde{\boldsymbol{w}}_{S_w^k} - \boldsymbol{w}_{S_w^k}\|_1 \tag{5}$$

Final objective function and algorithm

$$\begin{aligned} \min_{\widetilde{\boldsymbol{w}}} \mathbb{E}_x[D_1(y(x, \widetilde{\boldsymbol{w}}), y(x, \boldsymbol{w})) + \lambda_2 \mathcal{L}_{Topk}(\widetilde{\boldsymbol{w}}(x), \boldsymbol{w}(x)) \\ + \lambda_1 \max_{||\boldsymbol{\delta}|| \le R} D_2(y(x, \widetilde{\boldsymbol{w}}), y(x, \widetilde{\boldsymbol{w}} + \delta))] \end{aligned} \tag{6}$$

Where $\mathcal{L}_{Topk}(\widetilde{\boldsymbol{w}}(x), \boldsymbol{w}(x))$ is defined in (5).

We propose **Algorithm 1** to solve equation (6).

Algorithm 1 Finding a SEAT

Initialize $\tilde{\boldsymbol{w}}_0$.
for $t = 1, 2, \cdots, T$ **do**
  Initialize $\boldsymbol{\delta_0}$.
  for $p = 1, 2, \cdots, P$ **do**
    $\tilde{\boldsymbol{w}}'_{\boldsymbol{t-1}} = \tilde{\boldsymbol{w}}_{\boldsymbol{t-1}} + \boldsymbol{\delta_{p-1}}$
    Update $\boldsymbol{\delta}$ using PGD, where B_p is a batch
    $$\boldsymbol{\delta_p} = \boldsymbol{\delta_{p-1}} + \alpha_p \frac{1}{|B_p|} \sum_{x \in B_p} \nabla D_2(y(x, \tilde{\boldsymbol{w}}_{t-1}), y(x, \tilde{\boldsymbol{w}}'_{\boldsymbol{t-1}})).$$
    $\boldsymbol{\delta^*_p} = \arg\min_{||\boldsymbol{\delta}|| \le R} ||\boldsymbol{\delta} - \boldsymbol{\delta_p}||.$
  end for
  Update $\tilde{\boldsymbol{w}}$ using Stochastic Gradient Descent, where B_t is a batch
  $$\tilde{\boldsymbol{w}}_{\boldsymbol{t}} = \tilde{\boldsymbol{w}}_{\boldsymbol{t-1}} - \frac{\eta_t}{|B_t|} \sum_{x \in B_t} [\nabla D_1(y(x, \tilde{\boldsymbol{w}}_{\boldsymbol{t-1}}), y(x, \boldsymbol{w})) - \lambda_1 \nabla D_2(y(x, \tilde{\boldsymbol{w}}_{\boldsymbol{t-1}}), y(\tilde{\boldsymbol{w}}_{\boldsymbol{t-1}} + \boldsymbol{\delta^*_P})) - \lambda_2 \nabla \mathcal{L}_{Topk}(\boldsymbol{w}, \tilde{\boldsymbol{w}}_{t-1})].$$
end for
Return: $\tilde{\boldsymbol{w}}^* = \tilde{\boldsymbol{w}}_{\boldsymbol{T}}$.

Paper 2 Improving Faithfulness for Vision Transformers

The explanation modules should be sensitive enough to important token perturbations while remaining stable under noise perturbations.

Faithfulness Measurement

As attention mechanisms in ViTs outputs a vector indicating the importance of each visual token, **it's necessary to reconsider the robustness of attention module and ViTs.**

In general, faithful ViTs should satisfy:

1. The magnitude of each entry in the attention vector indicates the importance of its associated patch. To ensure interpretability robustness, it is sufficient to maintain the order of leading entries.

   We measure interpretability robustness by computing the overlap of the top-k indices between the **attention vector of the original input** and the **perturbed input**.

   Top-k: 原输入的注意力向量与扰乱的输入的 top-k indices 重叠度

2. The attention vector is fed to an MLP for prediction. **A robust attention vector should also preserve the final model prediction.** Specifically, we aim for the **prediction distribution based on the robust attention under perturbations is almost the same as the distribution without perturbation**.

   We measure the similarity or closeness between these distributions using different divergences.

   预测分布: 用 Renyi divergence (or distance)衡量扰乱与无扰乱的分布相似度

Can stable attention vectors' top-k indices imply predictions' robustness?

For instance, $v_1 = (0.1, 0.2, 0.5, 0.7)$ and $v_2 = (0.2, 0.8, 0.9, 2)$ have the same top-k (1,2,3,4). However, differences in **magnitudes can significantly affect final predictions**.

Therefore, besides top-k indices, **prediction distribution remaining relatively unchanged under perturbations is required to achieve robustness**.

We give the definition of top-k overlap ratio for two vectors:

Definition 4.1. For vector$x \in \mathbb{R}^n$, we define the set of top-k component $T_k(\cdot)$ as

$$T_k(x) = \left\{i: i \in [d] \; and \; \left\{\left|\{x_j \geq x_i : j \in [n]\}\right| \leq k\right\}\right\}.$$

And for two vectors x, x', their top-k overlap ratio $V_k(x, x')$ is defined as

$$V_k(x, x') = \frac{1}{k}|T_k(x) \cap T_k(x')|$$

Definition 4.2 Faithful ViTs. We call a function $f: \mathbb{R}^{q \times n} \longmapsto \mathbb{R}^n$ is an $(R, D, \gamma, \beta, k, \|\cdot\|)$-faithful attention module if for any input x, $x' \in \mathbb{R}^{q \times n}$ such that $\|x - x'\| \leq R$, $f(x')$ satisfies

1. Top-k Robustness: **overlap ratio of two vectors**

$$V_k\big(f(x'), f(x)\big) \geq \beta$$

2. Prediction Robustness: **similarities between two distributions**

$$D\big(\bar{y}(x), \bar{y}(x')\big) \leq \gamma$$

where $\boldsymbol{\bar{y}(x), \bar{y}(x')}$ **are the prediction distribution of ViTs** based on $f(x'), f(x)$ respectively.

We also call the vector $f(x)$ as an $(R, D, \gamma, \beta, k, \|\cdot\|)$-**faithful attention for** $\boldsymbol{x}$, and the models of ViTs based on f as faithful ViTs (FViTs).

1. $\boldsymbol{R}$ **represents the faithful radius**;
2. D is a similarity's metric for two distributions. $\boldsymbol{D}$ **can be a distance or a divergence**;
3. γ measures the closeness of the two prediction distributions $\bar{y}(x), \bar{y}(x')$;
4. $0 < \beta < 1$ is the robustness of top-k indices;
5. $\|\cdot\|$ is either the ℓ_2-norm or the ℓ_∞-norm (if x is a $d = q \times n$ dimensional vector).

Definition 4.3. Given **two probability distributions** P and Q, and $\alpha \in (1, \infty)$, the α-$\boldsymbol{Rényi}$ **divergence** $\boldsymbol{D}_\alpha(P \parallel Q)$ is defined as $D_\alpha(P \parallel Q) = \frac{1}{\alpha - 1} log\mathbb{E}_{x \sim Q}\left(\frac{P(x)}{Q(x)}\right)^\alpha$.

两个预测分布间的相似性用$\boldsymbol{Rényi}$ divergence 计算 其值小于$\boldsymbol{\gamma}$

Theorem 4.4. A function is ViTs' $(R, D_\alpha, \gamma, \beta, k, \|\cdot\|)$-faithful attention module, then if

$$\gamma \leq -\log\left(1 - p_{(1)} - p_{(2)} + 2\left(\frac{1}{2}\left(p_{(1)}^{1-\alpha} + p_{(2)}^{1-\alpha}\right)\right)^{\frac{1}{1-\alpha}}\right),$$

we have for all x' such that where $\|x - x'\| \leq R$,

$$argmax_{g \in \mathcal{G}}\mathbb{P}(\bar{y}(x) = g) = argmax_{g \in \mathcal{G}}\mathbb{P}(\bar{y}(x') = g)$$

where $\mathcal{G}$ is the set of classes, $p_{(1)}$ and $p_{(2)}$ refer to the largest and the second largest probabilities in $\{p_i\}$, where p_i is the probability that $\bar{y}(x)$ returns the i-th class.

Obtain Faithfulness

Denoised Diffusion Smoothing (DDS) leverages a smoothing-diffusion process to obtain faithful ViTs while also enhancing prediction performance.

DDS is **to obtain FViTs**. Its two main components:

1. **Standard randomized smoothing with Gaussian noise**
2. **Denoising diffusion probabilistic model**

We find **using randomized smoothing** to vanilla ViTs **and applying a denoised diffusion probabilistic model** to the perturbed input can adjust it to FViTs. Its corresponding attention module will become a faithful one.

For a given input image x, we preprocess it by adding some randomized Gaussian noise, $i.e.,\ \tilde{x} = x + z$ with $z \sim \mathcal{N}\left(0, \sigma^2 I_{q\times n}\right)$.

Then we denoise $\tilde{x}$ via denoised diffusion model to get $\tilde{x}$, and feed the perturbed-then-denoised $\tilde{x}$ to the **self-attention module $\boldsymbol{Z}$** and process to later parts of original ViTs to get predictions.

In total, we can represent the **attention module as** $\widetilde{w}(x) = Z\big(T(x+z)\big)$, where T is the denoised diffusion method.

Specifically, it has the following steps after we add Gaussian noise to x to get $\tilde{x}$.

In the first step, we establish a connection between the noise models utilized in randomized smoothing and diffusion models.

While randomized smoothing augments data points with additive Gaussian noise $i.e., x_{rs} \sim \mathcal{N}(x, \sigma^2 I)$, diffusion models rely on a noise model of the form $x_t \sim \mathcal{N}\left(\sqrt{\alpha_t}x, (1-\alpha_t)I\right)$, where the factor α_t is a constant derived from the timestamp t.

To employ a diffusion model for randomized smoothing, DDS scales $x_{rs}$by $\sqrt{\alpha_t}$ and adjusts the variances to obtain the relationship $\sigma^2 = \frac{1-\alpha_t}{\alpha_t}$.

Using this calculated timestep, we can then compute $x_{t^*} = \sqrt{\alpha_{t^*}}(x+\delta)$, where $\delta \sim \mathcal{N}(0, \sigma^2 I)$, and apply a diffusion denoiser on x_{t^*} to obtain an estimate of the denoised sample, $\hat{x} = denoise(x_{t^*}; t^*)$.

We repeat this denoising process multiple times (e.g., 100,000) to enhance robustness.

In the following, we will show **$\widetilde{\boldsymbol{w}}$ is a faithful attention module**.

First we provide notations. For input image x, we denote $\widetilde{w}_{i^*}$ as the i-th largest

component in $\widetilde{w}(x)$. Let $k_0 = [(1-\beta)k] + 1$ as the minimum number of changes on $\widetilde{w}(x)$ to make it violet the β-top-k overlapping ratio with $\widetilde{w}(x)$.

Let $\mathcal{S}$ denote the set of last k_0 components in top-k indices and the top k_0 components out of top-k indices.

Theorem 5.1. Consider the function $\widetilde{w}$ where $\widetilde{w}(x) = Z\big(T(x+z)\big)\ with\ Z^{\mathrm{T}} = self - attention(X) = softmax\left(\frac{Q^{\mathrm{T}}K}{\sqrt{q}}\right)V^{\mathrm{T}}W_L$, T as the denoised diffusion model and $z \sim \mathcal{N}\big(0, \sigma^2 I_{q\times n}\big)$. Then it is an $(R, D_\alpha, \gamma, \beta, k, \|\cdot\|_2)$-faithful attention module for ViTs for any $\alpha > 1$ if for any input image x we have

$$\sigma^2 \geq \max\{\frac{\alpha R^2}{2}(\frac{\alpha}{\alpha-1} In(2k_0\left(\sum_{i\in\mathcal{S}} \widetilde{w}_{i^*}^{\alpha}\right)^{\frac{1}{\alpha}} + (2k_0)^{\frac{1}{\alpha}} \sum_{i\notin\mathcal{S}} \widetilde{w}_{i^*}) - \frac{1}{\alpha-1} in(2k_0)), \frac{\alpha R^2}{2\gamma}\}$$

Theorem 5.1 indicates that $\widetilde{w}(x)$ will be faithful attention for input x when σ^2 is large enough. Equivalently, based on Theorem 5.1 and 4.4, we can also find a faithful region given β and k. Note that in practice it is hard to determine the specific α in $Rényi$ divergence. Thus, we can take the supreme w.r.t all $\alpha > 1$ in finding the faithful region.

Experiments of Models

Three benchmark datasets using ViT, DeiT, and Swin models

Project Code Framework

Final result: F(Perturbed text)=most similar text 和 original text(top-k)

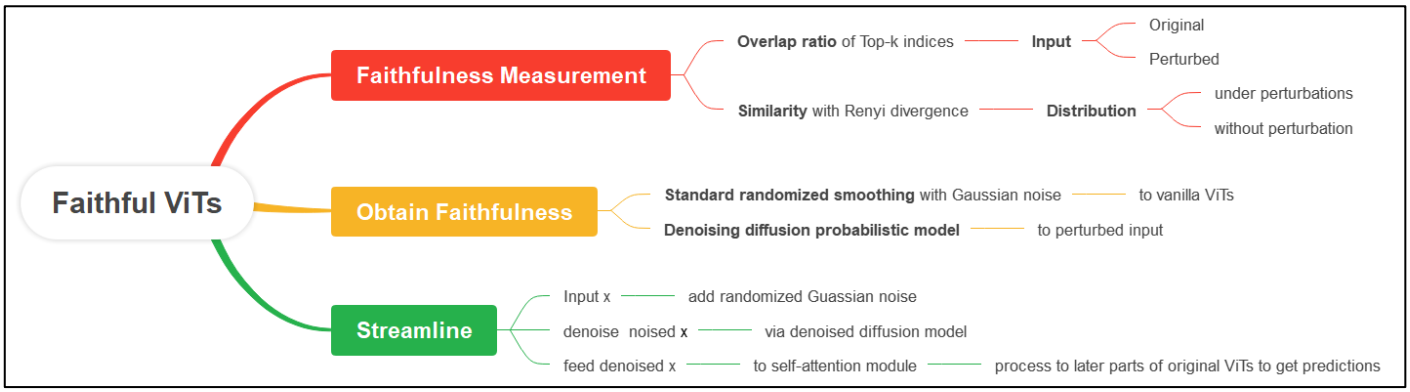
Model F(Perturbed Text from text fooler): text fooler 选对结果影响较大的同义词替换

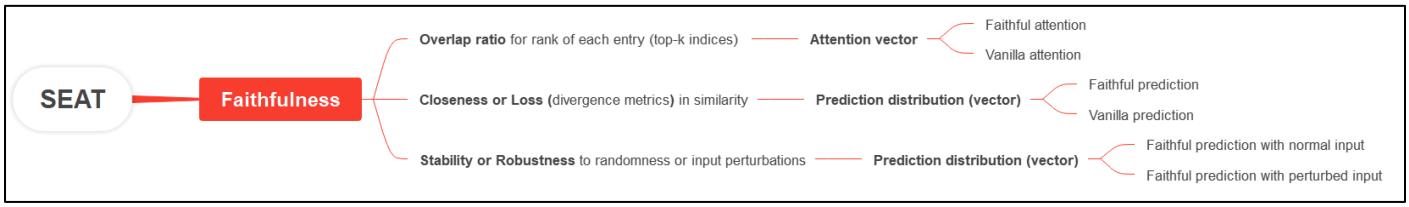
1. +z (embedding 里加一点点 Gaussian 数字)
2. 寻找 most similar text from glove embedding
3. Return most similar text

Original Text->Perturbed Text(staircase)->F_denoise (Perturbed Text)=most similar text

同义替换的方法：staircase

1. Method Validation: Bug，text fooler 模块(staircase)，跑 IMDB，结果（top-k）
2. Other methods: Reordering
3. Model: benchmark model
4. Dataset
5. Top-k 其他的指标来证明 这个 faithfulness：SEAT Reliability





Example of Input: Pre-processing

Input: [‘马和祝是好队友!’]

Tokenize: [‘马’, ’和’, ’祝’, ’是’, ’好’, ’队’, ’友’, ’!’]

索引化: [101, 11, 3, 30, 56,888] # 根据词表转换{‘[PAD]’:0, ’[CLS]’:101, ‘马’:11,⋯}

Padding: [101, 11, 3, 30, 56,888, 0, 0, ⋯, 0] # len=512

Embeddings: [0.0390, -0.0558,⋯,-0.0099] # len=512

Example of Input: Source Sequence

原始句子 0: 马畅很好. 工作很快.

模型输入 1:[**101**, 11, 3, 30, 22, **102**, 24, 7, 30, 24, **102, 0, 0, 0**]

模型输入 2:[0,0,0,0,0,0,1,1,1,1,1,0,0,0] # type ids 表明句子序号

模型输入 3:[F,F,F,F,F,F,F,F,F,F,F,T,T,T] # attention mask 向量

Example of Input: Q, K, V matrixes

Sentence: [Machine Learning Works] # 3 个词

Input: [Machine, Learning] # 输入前 2 个

Embedding: $X_1(Vector_{512*1}), X_2$

Queries: $X_1 * W^Q(Vector_{3*512}) \rightarrow q_1(Vector_{3*1}), X_2W^Q \rightarrow q_2$

Keys: $X_1 * W^K(Vector_{3*512}) \rightarrow k_1(Vector_{3*1}), X_2W^K \rightarrow k_2$

Values: $X_1 * W^V(Vector_{3*512}) \rightarrow v_1(Vector_{3*1}), X_2W^V \rightarrow v_2$

where matrix W^Q, W^K, W^V are fixed by training.

Input matrix: $[X_1, X_2]^T(Vector_{512*2})$# 矩阵分别乘$W^{Q,K,V}$得 2*3 的 q,k,v 矩阵 可并行运算

Example of input: Attention

假设词向量是 1*512, X 矩阵是 2*512, W^Q,K,V 均是 512*64(Head=8)

Input: $Q\ K\ V(Vector_{2*64})$

Step 1: $Score = Q * K^T(Vector_{2*2})$ # 2*3 的 Q 乘 3*2 的 K^T

Step 2: $Score_{d_k} = \frac{Score}{\sqrt{d_k}}$ (d_k=64 是 K 的维度)

Step 3: $Score_{softmax} = softmax(Score_{d_k})$ #分布在[0,1]之间的 2*2 概率分布矩阵

Step 4: $Attention = Score_{softmax} * V(Vector_{2*64})$

即:

$$\boldsymbol{Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{D_k}}\right)V}$$

Example of Encoder: Layer Normalization

	I	$\frac{Vector_{1*512}(64*8\text{Attention}) - u_1}{\sigma_1}$
T_x	Like	$\frac{Vector_{1*512}(64*8\text{Attention}) - u_2}{\sigma_2}$
	You	$\frac{Vector_{1*512}(64*8\text{Attention}) - u_3}{\sigma_3}$

Layer Norm 在一个隐藏层内部的不同值进行, 对于层 l, 记这一层的隐藏单元数为 H, 则 Layer Norm 可以描述为:

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l$$

$$\sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2}$$

此外, 还要设置参数 g 和 b 来允许网络重新设计隐藏层的方差和均值, 对于我们的隐藏层$A \in R^{[T_x,512]}$, 记这一层为 LayerNorm(A):

$$\mu = \frac{1}{T_x}\sum_{i=1}^{T_x} A_i \ \ (A_i \in R^{[1,512]})$$

$$\sigma = \sqrt{\frac{1}{T_x}\sum_{i=1}^{T_x}(A_i - \mu)^2}$$

$$LayerNorm(A) = \frac{\boldsymbol{g}}{\sigma} \odot (A - \mu) + \boldsymbol{b}$$

Example of Encoder: Feed-Forward Networks (FFN)

把$A_i \in R^{[1,512]}$使用 ReLU 激活函数的全连接运算得到一个长为 2048 的隐藏层向量:

$$A_i^{[1]} = ReLU(A_iW_1 + b_1) = \max\{0, A_iW_1 + b_1\}$$

式子中$W_1 \in R^{[512,2048]}$. 注意, 我们把$A_i \in R^{[1,512]}$都带入, 因此不同的$A_1, \dots, A_{T_x}$只是同一句话中不同单词对应的向量. 因此输出$A^{[1]}$大小是$[T_x, 2048]$.

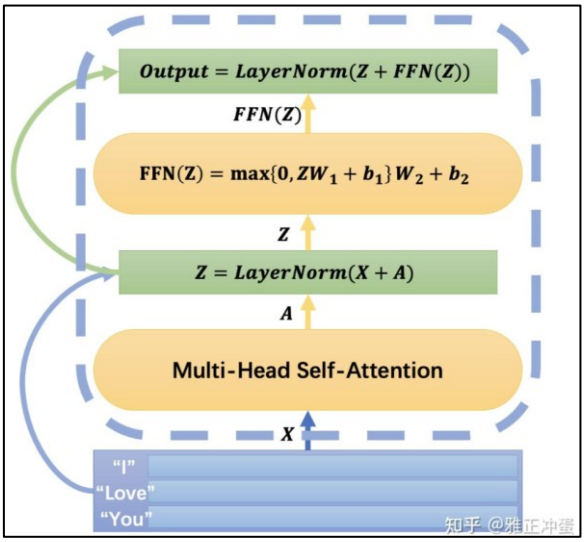
之后我们在使用一层线形层(不带激活函数)把向量大小变回$[T_x, 512]$即可, 因此我们的这个全连接层 FFN 就写作:

$$FFN(A) = \max\{0, AW_1 + b\}\, W_2 + b_2$$

上式中$W_2 \in R^{[2048,512]}$.

Example of Encoder: 残差连接(跳跃连接)

我们的 Encoder 模块因此有两个子模块，Multi-Head Self-Attention 层和 FFN 层. 这两个模块之间得到的输出还夹着残差连接和 LayerNorm 层.



Example of Jensen-Shannon Divergence (JSD)

我们使用 JSD 来为 attention without perturbation 和 attention under perturbation 之间评估可解释性的 stability.

$$JSD\ (P||Q) = \frac{1}{2}(KL(P||M) + KL(Q||M))$$

Where $M = \frac{1}{2}(P + Q)$ 且 KL 是 Kullback-Leibler divengence, 对于离散概率分布定义如下:

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

KL 是不同概率 P 和 Q 之间的 expectation of the logarithmic difference.

Explanation Streamline:

1. **Attention Weights:** Input of two sets of attention weights.
2. **Mean Distribution:** Compute the mean of two attention weight distributions
3. **KL Divergence:** Compute KL divergence between the mean distribution and each of the original distributions.
4. **JSD:** Compute the JSD as the average of the two KL divergences.
5. **Result:** Output the JSD value, indicating the similarity of two attention weights.

Example of Total Variation Distance (TVD)

我们把 TVD 用在预测分布之间.

$$TVD(P,Q) = \frac{1}{2}\sum_{I} |P_i - Q_i|$$

Explanation Streamline:

1. **Prediction Distributions:** Input two prediciton distributions
2. **Ensure Validity:** Ensure the distributions sum to 1
3. **Compute TVD:** Compute the TVD by taking the absolute diff between the corresponding elements of the two distributions, summing these diff, and then halving the result
4. **Result:** Output TVD value, indicating the diff between two prediction distributions

Example of Conprehensiveness, Sufficiency, Sensitivity

Comprehensiveness, Sufficiency (DeYoung et al. 2020). Following the notation in the previous study, we denote the model as $m(\cdot)$. Intuitively, model with higher interpretation ability should be less confident in making decision once the rationales are removed. Formally, we have

$$\text{comprehensiveness } = m\left(x_i\right)_j - m\left(x_i \backslash r_i\right)_j \tag{7}$$

Another related metric of comprehensiveness is sufficiency. It captures the degree that extracted rationales are sufficient for the model to make classification. Formally, we have

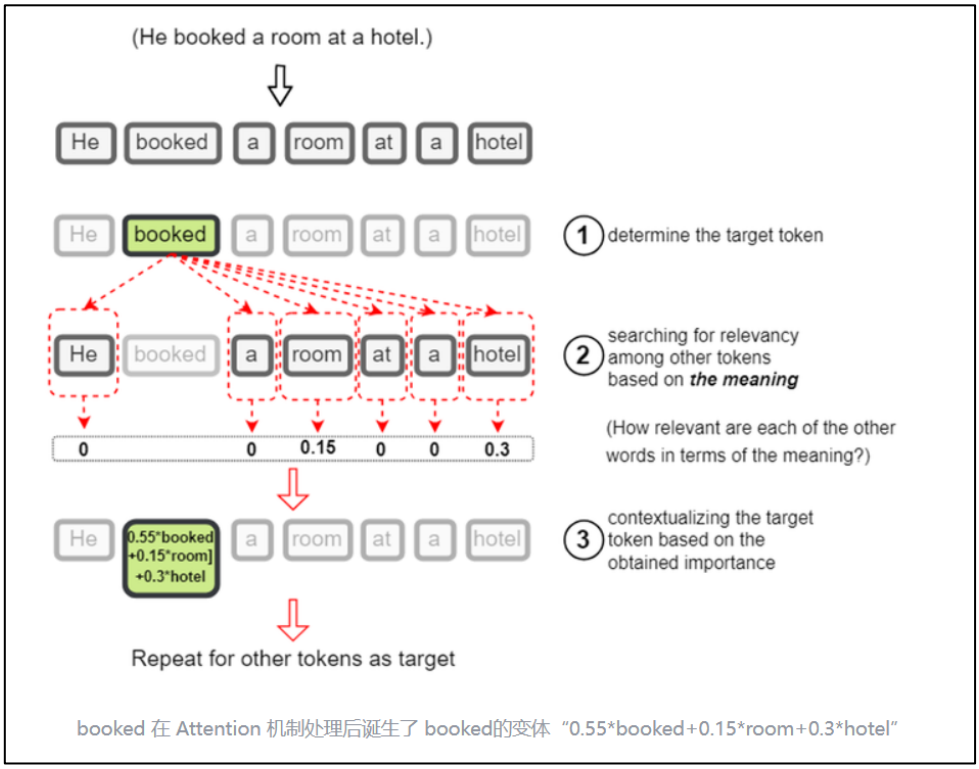
$$\text{sufficiency } = m\left(x_i\right)_j - m\left(r_i\right)_j \tag{8}$$

Sensitivity. (Yin et al. 2022) As point out in (Yin et al. 2022), models with higher interpretability should have lower sensitivity under some local and adversarial perturbations on important tokens. Specifically, given a sequence of rationales r_k, they evaluate the sensitivity by only adding perturbations to its corresponding embedding $e\left(r_k\right)$ and keeps the other token embedding the same as original. Then, they measures the minimal perturbation norm, denoted as ϵ_{r_k}, that changes the model prediction for this instance:

$$\epsilon_{r_k} = \min \|\boldsymbol{\delta}_{\boldsymbol{r_k}}\|_F \quad \text{s.t.} \quad f\left(e(x) + \boldsymbol{\delta}_{\boldsymbol{r_k}}\right) \neq y \tag{9}$$

where $\|\cdot\|_F$ is the Frobenius norm of the matrix, and $\boldsymbol{\delta}_{\boldsymbol{r_k}} \in \mathcal{R}^{n \times d}$ denotes the perturbation matrix where only the columns for tokens in r_k have non-zero elements. Given that the exact computation of ϵ_{r_k} is intractable, they use the PGD attack (Madry et al. 2018) with a binary search to approximate ϵ_{r_k}. Following (Yin et al. 2022), in practice, we vary the size of r_k, compute multiple ϵ_{r_k}, and summarize them with the area under the curve (AUC) score.

Attention Mechanism



Reference

[1] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[J]. 2017.DOI:10.48550/arXiv.1706.03762.

[2] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[J]. 2018.DOI:10.48550/arXiv.1810.04805.

[3] The Illustrated Transformer, http://jalammar.github.io/illustrated-transformer/.

[4] The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning) https://jalammar.github.io/illustrated-bert/.

[5]